Kaggle Competition: Facial Feature Classification Challenge

ENGE707 Data Engineering and AI

Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Dataset Exploration and Understanding **3**

1.1 Problem 3

1.2 Data Inspection 3

1.3 Number of Instances in each class 4

1.4 Class Image Inspection and Evaluation 5

1.5 Explanation of usage of Training, Validation and Kaggle Submission 6

2. Data Preprocessing and Pipeline Preparation **6**

3. Model Selection and Pipeline Development **7**

4. Training Strategy and Overfitting/Underfitting Management **7**

5. Model Evaluation and Kaggle Submission **8**

6. Results Analysis and Improvement Strategies **8**

7. Report Writing and Reproducibility **8**

1. Dataset Exploration and Understanding

1.1 Problem



The problem focuses on distinguishing between cats and dogs and which animal facial features (eye, nose, mouth or face) an image shows. This is a classification problem where there are 8 different classes: CatEyes, DogEyes, CatFace, DogFace, CatMouth, DogMouth, CatNose, or DogNose that the image can be classified as. If the model can succeed in providing a high accuracy and other metrics in the prediction results, the model could be used as the first step to further classify the breed and visual anomalies of the cat / dog. This prediction would allow for example a system to automatically describe what section of the animal is being checked and check the region for possible anomalies.

1.2 Data Inspection

```
Train rows, columns: (63660, 2)
Column dtypes: {'image_id': 'str', 'class': 'str'}
Null values: {'image_id': 0, 'class': 0}
Duplicate ids in CSV: 0
Train id range: 1 - 63660
Train file types: {'.jpg': 63660}

Test rows, columns: (29443, 2)
Duplicate ids in sample_submission: 0
Test id range: 1 - 29443
Test file types: {'.jpg': 29443}
Test classes in sample_submission: {'CatEye': 4567, 'DogEye': 4221, 'CatFace': 3571, 'CatMouth': 3570, 'CatNose': 3457, 'DogFace': 3407, 'DogMouth': 3341, 'DogNose': 3309}

Train files: 63660 | Test files: 29443
```

Figure 1.2.1 : Dataset Structure

The Kaggle competition provided testing and training folders as well as included the train_labels.csv which contains the mapping of the “image_id” and “class” columns, the “image_id” 00001 links to 00001.jpg in the train folder and the class is the target. The test folder contains the unlabelled testing images and the train folder contains the labelled training images (connected to the train_labels.csv).

In the training dataset, there are 63,660 rows and in the testing dataset there are 29,443 rows. The train_labels.csv consists of 2 columns “image_id” and “class”. With “image_id” as the unique identification of the image and class being the target.
In Figure 1.21, it showed that there are 0 duplicate ids in CSV and the id correctly ranges from 1 to 63660 which is the correct amount of images in the training folder.

The sample_submission.csv shows what the structure of the data submission to kaggle would need to be like with an unpadded “image_id” (e.g 00001 to 1) and the class column that describes the predicted target attribute.

1.3 Number of Instances in each class

	class	instances
0	CatEye	13884
1	CatFace	6942
2	CatMouth	6942
3	CatNose	6942
4	DogEye	11580
5	DogFace	5790
6	DogMouth	5790
7	DogNose	5790

Figure 1.3.1 : Class Distribution

In the train_labels.csv there, it shows that not all classes are distributed evenly. With CatEye (13,884) being the dominant class having more than twice the amount of instances than the other classes such as DogFace, DogMouth, and DogNose (5,790). This shows that there is a class imbalance in the training dataset. This would be considered during the preprocessed stage where the training and validation split.

1.4 Class Image Inspection and Evaluation



Figure 1.4.1 : Example Classification Images

The images shown of each class (CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth, and DogNose) all showed varying infomation. The images shown appeared to vary in resolution with some seemingly more pixelated than others, Some images do not clearly show the features such as the DogFace id17364 where it is hard to view the dog's face due to the fur color and clarity.

Additionally some of the images shown contain other classes that they weren't supposed to capture such as the DogNose id 27348 showed the Nose as well and 1 eye. This would make classification for models much more difficult due to the combination of multiple target classes.

The lighting and position of the images are also different as some appear overly bright and some appear dark. The head position of the cats and dog images are also not perfectly horizontal (e.g the CatNose Id 62280).

Image dimension summary:

	width			height		
	min	median	max	min	median	max
split						
test	8	82.0	1832	9	80.0	1978
train	4	84.0	2219	4	80.0	2219

Figure 1.4.2 : Image Dimension Summary

Further confirming the dimension of the images, Figure 1.4.2 shows the summary of all the images, with the train images ranging from 4px to 2219px and the height ranging from 4px to 2219px. This range is apparent for the test images as well.

```
File-reading or decoding failures: 0
```

Figure 1.4.3 : Checking Corrupted Files

Confirmed that there are no corrupted files in both the testing image folder and the training image folder.

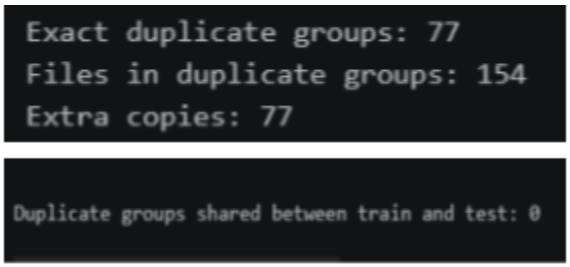


Figure 1.4.4 : Checking Duplicates

There are 77 exact duplicate groups from using SHA 256 hash in both the testing folder and the training folder. This means that there are 77 different groups of images, and 2 images have the same hash. Though there are duplicates in both the testing and training folder, there are no shared duplicates between those folders. Therefore to prevent data leakage, the duplicates would need to be kept in the same dataset for training or validation. This checks only if the byte content is the same, it does not check if the image was resized, or recompressed and each image has not been checked visually if they are similar. Further evaluation would proceed in the preprocessing stage whether duplicates would need to be removed.

1.5 Explanation of usage of Training, Validation and Kaggle Submission

For the dataset, the test dataset (images) would not be used in training in any way, only used for the testing for the final kaggle submission. The training dataset (images) would be applied to the 80 / 20 split so that 80% of the data would be for training and the 20% would be for validation. Stratified splitting would be applied to ensure that the class proportion is the same for both the training and the validation subsets. The 80% training data would be used to actually learn the pattern from images to adjust appropriate weights for each classification and the validation dataset is used to evaluate the performance on unseen data, so that further improvement could be made. Once the performance is satisfactory the trained deep learning model would be used on the test dataset to predict all instances in the dataset, the result of the prediction would be saved in a csv file to be submitted to kaggle.

2. Data Preprocessing and Pipeline Preparation

- [ ] Students must prepare the image data and develop a reproducible preprocessing pipeline. Students are required to:
    - [ ] Resize images to a suitable input size for the selected model.
    - [ ] Normalise or rescale pixel values appropriately.
    - [ ] Encode class labels into a suitable numerical format.
    - [ ] Split the labelled training data into training and validation sets using a justified strategy.
    - [ ] Apply appropriate data augmentation techniques where relevant.
    - [ ] Use suitable data loaders, generators, or dataset objects for efficient training.
    - [ ] Ensure that preprocessing is applied consistently to training, validation, and test images.
    - [ ] Avoid data leakage between training, validation, and test data.

2.1 Resize Image

3. Model Selection and Pipeline Development

- [ ] Students must design and implement a complete deep learning image classification pipeline. Students are required to:
    - [ ] Select and justify a suitable deep learning architecture.
    - [ ] Use a custom CNN, transfer learning model, Vision Transformer, hybrid CNN--Transformer model, or another appropriate deep learning model.
    - [ ] Describe the model architecture and its main components.

- [ ] Specify the input image size, output layer, activation function, and number of target classes.
- [ ] Select and justify the loss function, optimiser, learning rate, batch size, and number of epochs.
- [ ] Implement the complete training pipeline in Python.
- [ ] Ensure that the pipeline can generate predictions for the Kaggle test set.

4. Training Strategy and Overfitting/Underfitting Management

- [ ] Students must train their model and monitor its learning behaviour. Students are required to:
  - [ ] Train the selected model using the prepared training data.
  - [ ] Monitor training and validation accuracy and loss.
  - [ ] Identify signs of overfitting or underfitting.
  - [ ] Apply appropriate strategies to improve generalisation, such as data augmentation, dropout, weight decay, batch normalisation, early stopping, learning-rate scheduling, transfer learning, fine-tuning, or model adjustment.
  - [ ] Present and discuss learning curves for training and validation performance.
  - [ ] Justify any changes made to the model or training strategy.

5. Model Evaluation and Kaggle Submission

- [ ] Students must evaluate their model and submit predictions to the Kaggle competition. Students are required to:
  - [ ] Evaluate the model on the validation set using suitable classification metrics.
  - [ ] Report validation accuracy and loss.
  - [ ] Include additional metrics where appropriate, such as precision, recall, F1-score, confusion matrix, or class-level accuracy.
  - [ ] Generate predictions for the Kaggle test images.
  - [ ] Prepare the submission file in the required Kaggle format.
  - [ ] Submit predictions to the Kaggle competition.
  - [ ] Report the Kaggle leaderboard score and include evidence of the submission.
  - [ ] Compare validation performance with Kaggle leaderboard performance.

6. Results Analysis and Improvement Strategies

- [ ] Students must critically analyse the results and discuss possible improvements. Students are required to:

- [ ] Interpret the model performance using validation results and Kaggle score.
- [ ] Analyse common misclassifications and difficult classes where possible.
- [ ] Discuss possible reasons for performance limitations.
- [ ] Evaluate the effectiveness of preprocessing, augmentation, model choice, and training strategy.
- [ ] Compare attempted model versions or experimental settings where applicable.
- [ ] Propose realistic strategies for improving performance, such as alternative architectures, hyperparameter tuning, improved augmentation, fine-tuning, ensemble methods, or better validation strategies.
- [ ] Reflect on the strengths and weaknesses of the final solution.

7. Report Writing and Reproducibility

- [ ] Students must prepare a clear individual technical report and provide reproducible supporting files. Students are required to:
    - [ ] Write a structured final report of 3,000- 3,500 words, describing the complete workflow.
    - [ ] Include sufficient detail about the dataset, preprocessing, model, training strategy, evaluation, Kaggle results, analysis, and improvement strategies.
    - [ ] Provide source code or a Jupyter Notebook with clear comments and instructions.
    - [ ] Include the final Kaggle submission file.
    - [ ] Include Kaggle username and leaderboard evidence.
    - [ ] Reference all external libraries, models, papers, tutorials, or resources used.
    - [ ] Ensure that the work is original and complies with academic integrity requirements.
    - [ ] Submit a Turnitin similarity report with a similarity score not exceeding 20%.